



BTS-SIO

Conception et développement d'applications

Projet BTS – client Rich

**AG'IMMO – JAVAFOX
LOT-1**



Objectifs

Le développement informatique concerne tout ce qui touche à l'étude, à la conception, à la construction, au développement, à la mise au point, à la maintenance et à l'amélioration des logiciels et autres applications et sites web. C'est le développeur informatique qui en a la charge. Il a plusieurs rôles : analyser les besoins des clients/utilisateurs, s'occuper de l'écriture informatique, rédiger les notices...etc.

Il doit posséder de multiples compétences : connaissance du langage informatique, expertise des technologies de bases de données.

JavaFX est un Framework et une bibliothèque d'interface utilisateur issue du projet OpenJFX, qui permet aux développeurs Java de créer une interface graphique pour des applications de bureau, des applications internet riches et des applications smartphones et tablettes tactiles.

Le passage du BTSSIO option SLAM implique de mettre en œuvre une application type client riche.

Le client riche permet de développer des applications traditionnelles type client lourd, ou des applications type client-serveur. Il propose :

un Framework de développement et des composants de base pour faciliter le travail des développeurs incluant notamment des composants IHM enrichis ;

un environnement d'exécution à installer sur les postes clients, comprenant des composants de base sur lequel seront déployées les applications ; typiquement l'environnement d'exécution Java (JRE) pour les applications écrites en langage Java.

L'application client riche qui sera développée est une application de gestion d'une agence immobilière.



Contrainte de cet atelier

Connaitre correctement les bases du langage Java et la POO.

Avoir déjà vu les bases du Framework **JavaFx**

Considérations techniques & logicielles

Eclipse IDE pour les phases de programmation.

JavaFx installé et configuré sous Eclipse



Bloc

Bloc de compétences n°2 : option B « Solutions logicielles et applications métiers » - Conception et développement d'applications



Titre

Ag'Immo javaFX – lot1



Historique des modifications

Date	Nom	Description de la version	Version
2023-09-01	Acasara	Création du document	V.1
		-	

Table des matières

1. DÉVELOPPEMENT D'UN LOGICIEL DE GESTION IMMOBILIERE	4
1.1. Présentation de l'entreprise : Ag'immo	4
1.1.1. Produits et/ou services vendus.....	4
1.2. Analyse de l'existant	4
1.2.1. Quels sont les problèmes rencontrés actuellement ?.....	4
1.3. La demande	4
1.3.1. Les objectifs	4
1.3.2. Les cibles	5
1.4. DÉVELOPPEMENT DU LOGICIEL	5
1.4.1. Caractéristiques et fonctionnalités	5
1.4.2. Structure du logiciel	5
1.4.3. Structure du développement.....	5
1.5. CHARTE GRAPHIQUE DU LOGICIEL.....	6
1.5.1. Le logo	7
1.5.2. La typographie	7
1.5.3. Les couleurs	8
1.5.4. La normalisation de la mise en forme.....	8
2. Lot 1 de l'application.....	10
2.1. Création du projet.....	10
2.3. fichier properties	11
2.4. Fourniture des éléments de départ.....	11
2.5. La base de données	12
3. Description des différents éléments.....	13
3.1. Classe LoginController.java	13
3.2. Classe DashboardController.java	13
3.3. Classe AgentDefinitionController.java.....	15
3.4. Installation des jars pour l'édition des cartes de visite	16
4. Livraison du lot 1 pour tests	18

1. DÉVELOPPEMENT D'UN LOGICIEL DE GESTION IMMOBILIERE

1.1. Présentation de l'entreprise : Ag'immo

Société à responsabilité limitée est en activité depuis 9 ans. Localisée à GRENOBLE (38100). Elle possède trois agences :

- Grenoble (siège administratif)
- Villard de Lans
- St-Marcellin

- **Date de création :**

01/01/2015 par Benjamin Clair qui est gérant de l'entreprise

- **Activité principale :**

elle est spécialisée dans le secteur d'activité de la location et la vente de biens immobiliers.

- **Nombre de salariés :**

10 salariés au 01/09/2024

- **Chiffre d'affaires :**

1 250 000 € en 2023

1.1.1. Produits et/ou services vendus

La mission de **Ag'immo** est de vous assister, vous conseiller et vous guider en respectant vos choix et impératifs, dans le monde de l'immobilier.

Ag'immo va gérer vos biens immobiliers, les administrer (états des lieux, perception de loyer, relances, ...etc.) .

Elle gèrera également la vente de votre bien en organisant les visites, les rencontres avec les acheteurs potentiels.

nous vous proposons des solutions clés en main.

Pas de facturation supplémentaire, tout est clair.

Notre objectif : mériter et conserver votre confiance pour des échanges gagnant/gagnant.

Conformément aux dispositions du Code de la consommation concernant « *le processus de médiation des litiges de la consommation* », le client a le droit de recourir gratuitement au service de médiation proposé par Ag'immo. Le médiateur "droit de la consommation" ainsi proposé est MEDIAVET.

1.2. Analyse de l'existant

1.2.1. Quels sont les problèmes rencontrés actuellement ?

La gestion des différents acteurs et biens présents et gérés par l'agence se fait au travers d'une application open-source mise en place en 2015 lors de la création de l'entreprise.

Cette application a depuis un certain temps montré ses limites, que ce soit en volume mais aussi en capacités d'évolution. Pour pallier à ces problèmes de nombreux agents ont recours à des outils «*personnels*» (logiciels téléchargés, tableurs, ...etc.).

1.3. La demande

1.3.1. Les objectifs

Il a été décidé, après le bilan de l'année comptable fin Mai 2024, d'embaucher une équipe informatique afin de développer une application spécifique plus proche des attentes des utilisateurs.

Cette application devra permettre d'avoir des informations centralisées au sein de chaque agence (chacune gérant son propre portefeuille de biens et de locataires / propriétaires).

Les données présentes étant d'ordre privé et financier, la direction ne souhaite pas se tourner vers une application de type Web avec un stockage dans le Cloud. Ce type d'application sera développée, dans un second temps, pour proposer une vitrine de l'agence .

1.3.2. Les cibles

L'application sera à l'usage unique des agents des différentes agences ainsi que la direction.

Cette application devra permettre de gérer les différents acteurs et biens présents dans les agences.

Un site de présentation sous WordPress et géré par une société tierce existe déjà et est mis à jour quotidiennement. Il sera remplacé ultérieurement.

1.4. DÉVELOPPEMENT DU LOGICIEL

1.4.1. Caractéristiques et fonctionnalités

Dans sa version finale, le logiciel devra permettre de gérer l'intégralité des données utiles et obligatoires pour la gestion d'une agence immobilière.

Fonctionnalités du logiciel – Liste non exhaustive

- Gestion des Locataires
- Gestion des locations
- *Gestion des biens mis en vente*
- *Appels de loyers*
- *Encaissement des loyers*
- *Gestion des impayés*
- *Gestion des quittances*
- *Gestion des charges*
- Gestion des baux
- *Déclarations fiscales, ...etc.*

Contraintes du logiciel

- Le logiciel doit gérer des droits d'accès ;
- Le logiciel doit avoir un système d'authentification ;
- La base de données sera sur un serveur distant, sous un système reconnu, sécurisé et professionnel.
- Le logiciel doit être développé dans un mode écrans Windows (Client/serveur), portable sur Linux ou MacOS si besoin.

1.4.2. Structure du logiciel

La structure complète du logiciel sera plus explicitement présentée dans les différents documents d'analyse fonctionnelle (*les différents lots de l'application*).

1.4.3. Structure du développement

Le développement de l'application se fera en termes de lot ayant une caractéristique propre, une durée de développement définie, des rendus à fournir afin de tester l'application au fur et à mesure de son évolution. Une première étude a permis de définir, dans les grandes lignes, les différents lots :

- Lot 1 : base de départ de l'application. Tests de connexion, gestion de la page de login, du tableau de bord, des droits, sécurité et de la mise à jour des données de l'agent connecté. Simple impression des cartes de visite de l'agent ;
- Lots 2 & 3 : sécurisation et traitements de généralisations des opérations sur la base de données et les différentes fenêtres. Mise en place d'un historique des modifications de la base de données ;
- Lot 4 : gestion des propriétaires ;
- Lot 5 : gestion des locataires et propriétaires ;
- Lot 6 : gestion des locations ;
- Lot 7 : gestion des baux de location, des états des lieux ;
- Lot 8 : gestion des indicateurs sur le tableau de bord (*fin de bail, rdv, etc.*). gestion des rendez-vous d'un agent.
- Lot 9 : gestion des passerelles entre les différentes options (*les locations d'un propriétaire, l'historique des locataires, etc.*)
- Lot 10 : gestion des éditions de l'application.

Les informations sur un lot (*durée, caractéristiques, etc.*) vous seront données au fur et à mesure des analyses.

Ces lots représentent la version 1 de l'application. Cette version permettra de gérer simplement les locations et les locataires.

La version 2 traitant de la gestion des données fiscales sera développée après la mise en production de la version 1.

La version 3 traitant des biens en ventes sera développée après la mise en production de la version 2.

Ces deux versions ne sont ni analysées, ni chiffrées pour l'instant.

1.5. CHARTE GRAPHIQUE DU LOGICIEL

Une charte graphique représente un document qui regroupe la totalité des codes et des règles fondamentales qui définissent une application.

Bien qu'ayant plusieurs objectifs distincts, la fonction principale de la charte est d'identifier, dans notre cas, l'application. Ainsi, cette dernière sert à :

- Identifier facilement l'image et la personnalité d'un projet, d'une organisation ou d'une entreprise ;
- Assurer une cohérence et une harmonisation sur l'ensemble des supports de communication utilisés ;
- D'implanter cette image à long terme ;

La charte graphique est essentielle à toute entreprise car elle définit l'intégralité de son identité graphique par le biais de différents éléments.

1.5.1. Le logo

Le logo est le composant principal d'une identité graphique. Dans le cas du logo, la charte intervient afin d'éviter toute mauvaise utilisation de ce dernier (*trop étiré, déformé, de la mauvaise couleur, etc.*). Elle définit les utilisations à respecter quant à son positionnement, à ses proportions, à ses marges avec les autres éléments d'un document, etc.

Pour l'application **AgImmo**, le logo, quel que soit la plateforme, sera le suivant :



1.5.2. La typographie

Une charte permet, d'autre part, de définir la typographie (*police de caractères*) à utiliser dans une application. La police peut varier d'un titre à un corps de texte permettant de créer différents niveaux de texte, optimisant ainsi le confort de lecture. Une fois définie dans la charte graphique d'une entreprise, la ou les typographies choisies s'appliquent systématiquement sur l'ensemble des supports.

Polices de caractères :

Deux polices seront utilisées : Arial et Times New Roman

Messages à l'utilisateur : Arial 12px Bold Italic (*couleur rouge si message d'erreur*)

Zones de saisie : Arial 14px

Titres : Times New Roman 36px Bold Italic

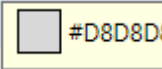
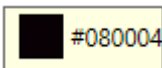
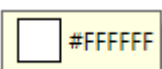
Label : Arial 14px ou 20px

Boutons : Arial 12px

1.5.3. Les couleurs

Généralement limitées, les couleurs offrent un atout de mémorisation considérable. La charte graphique indique quelles sont les applications des couleurs autorisées, dans le cadre d'une communication imprimée ou en ligne.

Couleurs de l'application :

- Rouge bordeaux : #8A0808 
- Gris clair : #D8D8D8 
- Gris foncé : #667882 
- Noir : #080004 
- Blanc : #FFFFFF 

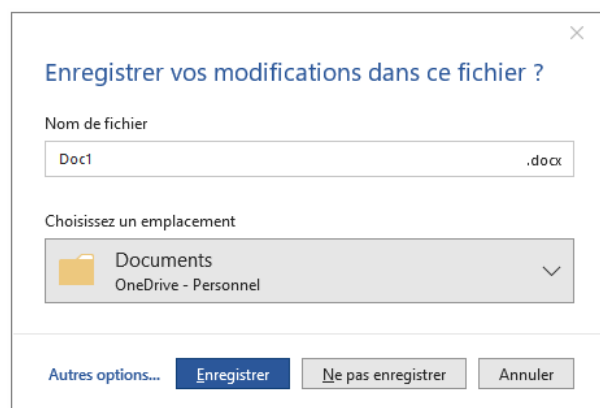
1.5.4. La normalisation de la mise en forme

Les menus seront systématiquement positionnés en haut d'une fenêtre ;



Les boutons ?

Le bouton valider doit-il être placé à droite ou à gauche du bouton annuler? Comment mettre en valeur les actions principales par rapport aux actions secondaires? Comment inciter les utilisateurs à effectuer les actions adéquates?



Avant de prendre une décision, l'utilisateur scanne toutes les possibilités en lisant les intitulés des boutons d'action.

Le positionnement optimal est donc d'ordonner l'action secondaire et l'action principale en fonction du sens de lecture de l'utilisateur et d'ainsi limiter la charge cognitive.

Les boutons de traitements d'une fenêtre de définition seront systématiquement positionnés en bas et à droite avec ces normes visuelles.



Les boutons de traitements d'une fenêtre de gestion seront systématiquement positionnés en bas et à droite avec ces normes visuelles.



2. Lot 1 de l'application

Vous reprenez à votre charge le développement de l'application. Une partie du lot a été faite notamment au niveau de l'IHM. Les fichiers FXML sont donc déjà tous définis. Les contrôleurs, dans leur définition aussi.

Il convient néanmoins de contrôler que les différentes liaisons entre fichier FXML et contrôleur soient correctement définis.

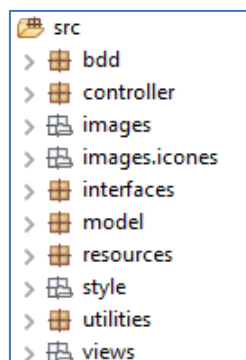
Le premier travail que vous aurez à faire va être de remettre l'application *sur pieds !!* Vous allez devoir positionner les différents éléments dans leur package respectifs, analyser si tout est conforme aux règles de nommage d'une application javaFX puis développer les parties manquantes.

Il va vous falloir, également, étudier l'application pour en comprendre le fonctionnement et la logique. Ce projet doit également être géré sous Git par le groupe.

2.1. Création du projet

Sous Eclipse :

- Créez un nouveau projet [JavaFx Project] .
- Nommez-le :
 - **AgImmo**
- Rajoutez-les [*Users Librairies*] **JavaFx** et **SQLServer**
- Supprimer les différents éléments inutiles (*Main.java, application.css, etc.*)
- Créez les différents packages nécessaires à l'application :
 - **bdd** : classes d'accès à la base de données ;
 - **controller** : classes contrôleurs des fenêtre FXML ;
 - **images** : images de l'application ;
 - **images.icons** : icones de l'application ;
 - **interfaces** : interfaces de l' application ;
 - **model** : classes modèle ;
 - **resources** : constantes et properties ;
 - **style** : feuilles de style ;
 - **utilities** : classes utilitaires (*contrôles, etc.*) ;
 - **views** : fichiers FXML.



2.3. fichier properties

Les **properties** en Java sont des valeurs de configuration que nous allons utiliser dans l'application pour stocker les informations de connexion à la base de données entre autres. Un fichier de propriétés est un fichier qui contient ces valeurs dans des paires clé-valeur. chaque ligne d'un fichier **.properties** comporte une seule valeur.

1. récupérez le fichier **conf.properties** et placez le dans le package **resources** ;
2. Adaptez-le à votre configuration.



```
#=====
# fichier      : conf.properties
# Package      : resources
# Version      : V1.0
#
# Description   : Fichier contenant les différentes données utiles à l'application.
#=====
# Données connexion SQLServer
jdbc.driver.class=com.microsoft.sqlserver.jdbc.SQLServerDriver
jdbc.url=jdbc:sqlserver://127.0.0.1;databaseName=AgImmo
jdbc.login=AdmAgImmo
jdbc.pwd=azerty1!
```

Pour pouvoir accéder à ces propriétés, récupérez la classe **UtilitiesProperties.java** et placez la dans le package **utilities**.

Cette classe possède une méthode :

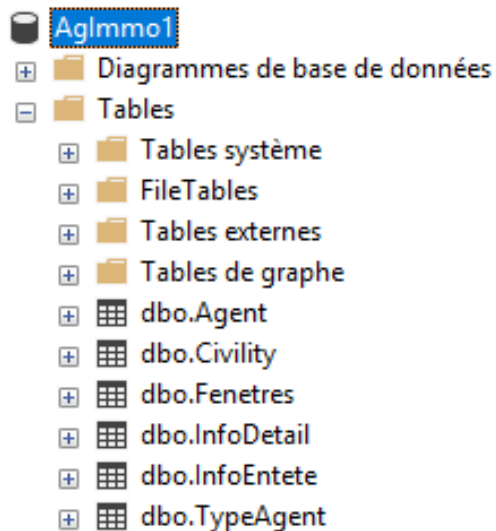
```
public static String getConfigProperties(String strProperty) {
permettant de récupérer la valeur correspondant à une clé.
```

2.4. Fourniture des éléments de départ

Récupérez les différents éléments de l'application et positionnez-les dans les bons packages.

2.5. La base de données

La base de données a la structure suivante :



Vous trouverez le script SQL de création des différents éléments de la base de données dans le dossier d'installation.

La base de données devra être accessible via dBiever.

3. Description des différents éléments

3.1. Classe LoginController.java

Cette classe est le contrôleur de la fenêtre de login de l'application.

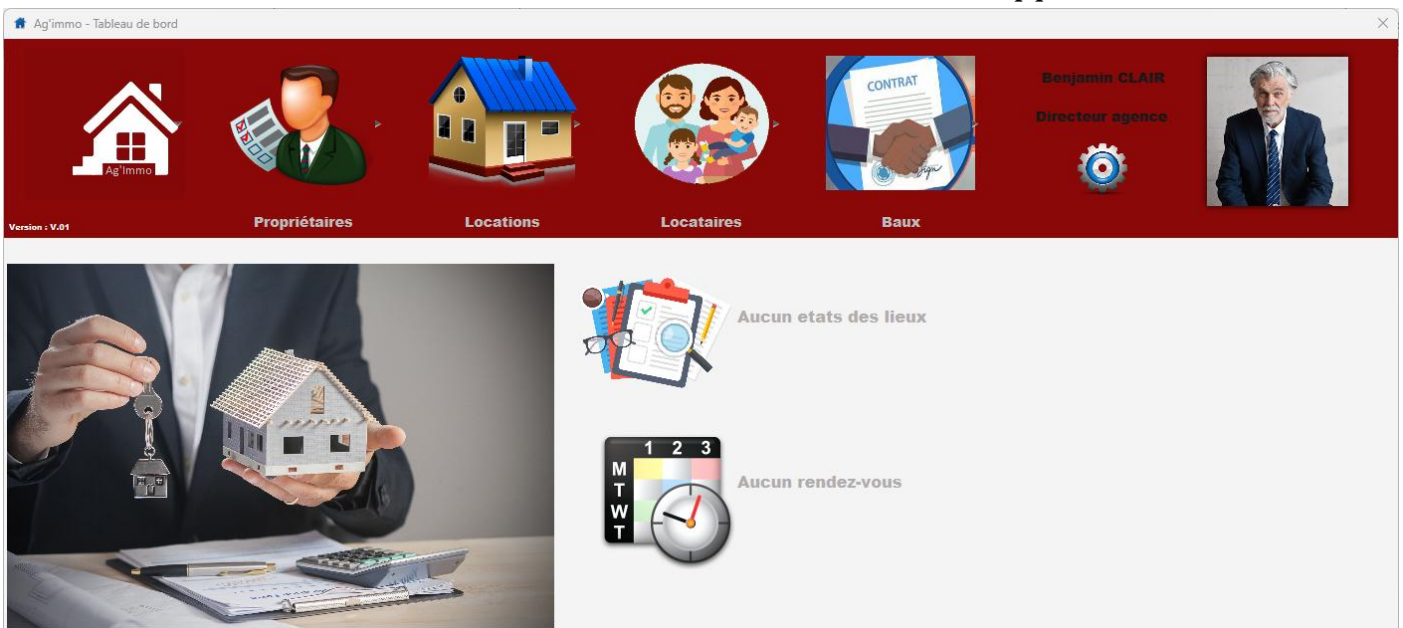


Travail à faire :

1. La zone erreur doit être invisible à l'initialisation ;
2. Un clic sur le bouton Valider ou le fait d'appuyer sur la touche Entrée permet de gérer la connexion ;
3. Le contrôle doit se faire sur le couple login/ mot de passe dans la base de données ;
4. Si la saisie est incorrecte, les deux zones sont vidées et la zone erreur est affichée ;
5. Si la saisie est correcte, appel du tableau de bord ;
6. Passage de l'agent connecté au tableau de bord.

3.2. Classe DashboardController.java

Cette classe est le contrôleur de la fenêtre tableau de bord de l'application.



Travail à faire :

1. La version de l'application doit être initialisée (texte + valeur classe Cstes) ;

2. Les informations de l'agent connecté doivent apparaître (*nom, prénom, type et photo*).
3. Seule option possible, modification de l'agent connecté en cliquant sur le .
Cet évènement correspond à la méthode **evtOnMouseClickedInvParametresAgent ()**;
4. Les autres actions ont leurs codes.

3.3. Classe AgentDefinitionController.java

Cette classe est le contrôleur de la fenêtre de définition de l'agent connecté.

Ag'immo - Données de l'agent

Modification d'un agent

Monsieur *
CLAIR *
Benjamin *
 06-16-88-02-61 *
 04-76-22-02-61 *
 benjamin.clair@yahoo.fr *
Directeur agence
bclair
..... *
Confirmation du mot de passe
 Choisir une image CLAIR Benjamin.png
Dans le dossier : E:/Documents/AgImmo/Agents
 Valider Annuler

Travail à faire :

1. Initialiser les différentes zones avec les données de l'agent connecté ;
2. Si validation :
 - a. Les zones avec une étoile sont obligatoires ;
 - b. Le téléphone et portable ont un format défini :

06-16-88-02-61 *
 Le n° doit avoir le format suivant : XX-XX-XX-XX-XX

- c. L'adresse mail a un format obligatoire avec @ et . ;

benjamin.clair@yahoo.fr *
Le mail doit avoir le format suivant : XXXXXXXX@YYYY.ZZZ

- d. Le mot de passe doit avoir des règles spécifiques :

..... *
Confirmation du mot de passe
 Choisir une image CLAIR Benjamin.png
Dans le dossier : E:/Documents/AgImmo/Agents
 Le mot de passe doit contenir au moins :
- un chiffre entre 0 et 9
- une lettre en minuscule
- une lettre en majuscule
- un caractère spécial
- une longueur entre 8 et 20

- e. Le mot de passe doit être identique à la confirmation ;
- f. L'agent doit avoir une image.

Une fois que les différents auront été effectué, s'il y a des erreurs, il faut afficher ces erreurs dans une dialogBox.

S'il ne reste pas d'erreur, il faut créer une nouvelle instance de l'agent et la «renvoyer» au tableau de bord afin qu'il se mette à jour.

1. Le nom doit être stocké dans la base de données en majuscule
2. Le login doit respecter les règles suivantes :
 - a. Première lettre du prénom en minuscule
 - b. Le nom en minuscule (si le nom est en plusieurs parties, il faut enlever les espaces) exemple : Benoit LE BRETON → blebreton
3. Lors de la validation, contrôlez que le login respecte bien ces règles
4. Rajoutez deux évènements (codes en Annexe) permettant de mettre à jour la zone Login chaque fois que le nom ou le prénom change.

3.4. Installation des jars pour l'édition des cartes de visite

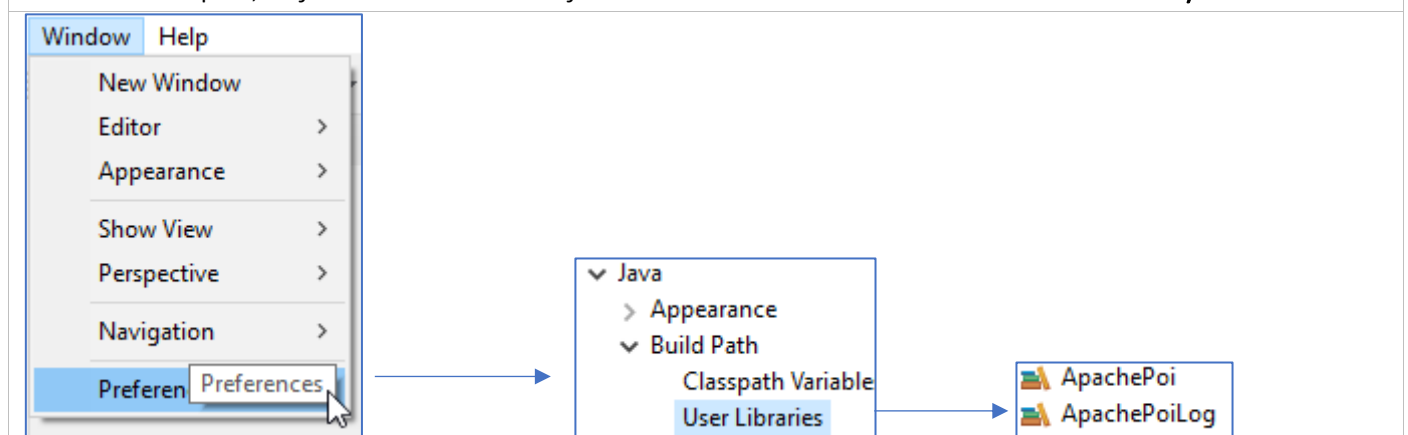
Pour pouvoir éditer les cartes de visite de l'agent, vous avez besoin de fichiers jars supplémentaires. Nous allons utiliser Apache POI (*POI est un projet de l'Apache Software Foundation permettant de manipuler avec la technologie Java divers types de fichiers créés par Microsoft Office*)



Partie
1

Récupérez les dossier ApachePOI

1. Récupérez dans Ressources, deux dossiers : **ApachePOI** et **ApachePOILog** ;
2. Copiez ces deux dossiers dans notre dossier **C:\Program Files\Java** ;
3. dans Eclipse, rajoutez les fichiers jar des deux dossiers comme **User Library**.



 ApachePoi

- >  commons-codec-1.15.jar - C:\Program Files\Java\ApachePoi
- >  commons-collections4-4.4.jar - C:\Program Files\Java\ApachePoi
- >  commons-compress-1.21.jar - C:\Program Files\Java\ApachePoi
- >  commons-io-2.11.0.jar - C:\Program Files\Java\ApachePoi
- >  commons-logging-1.2.jar - C:\Program Files\Java\ApachePoi
- >  commons-math3-3.6.1.jar - C:\Program Files\Java\ApachePoi
- >  log4j-api-2.18.0.jar - C:\Program Files\Java\ApachePoi
- >  poi-5.2.3.jar - C:\Program Files\Java\ApachePoi
- >  poi-excelant-5.2.3.jar - C:\Program Files\Java\ApachePoi
- >  poi-javadoc-5.2.3.jar - C:\Program Files\Java\ApachePoi
- >  poi-ooxml-5.2.3.jar - C:\Program Files\Java\ApachePoi
- >  poi-ooxml-full-5.2.3.jar - C:\Program Files\Java\ApachePoi
- >  poi-ooxml-lite-5.2.3.jar - C:\Program Files\Java\ApachePoi
- >  SparseBitSet-1.2.jar - C:\Program Files\Java\ApachePoi
- >  xmlbeans-5.1.1.jar - C:\Program Files\Java\ApachePoi

 ApachePoiLog

- >  log4j-core-2.18.0.jar - C:\Program Files\Java\ApachePoiLog

4. rajoutez ces deux **User Librairy** Dans le BuildPath de votre projet

 Modulepath

- >  ApachePoi
- >  ApachePoiLog
- >  JavaFX
- >  JavaFX SDK
- >  JRE System Library [JavaSE-20]

4. Livraison du lot 1 pour tests

Date de livraison (V1)

La date de livraison est le 20 décembre 2024 – 17h30

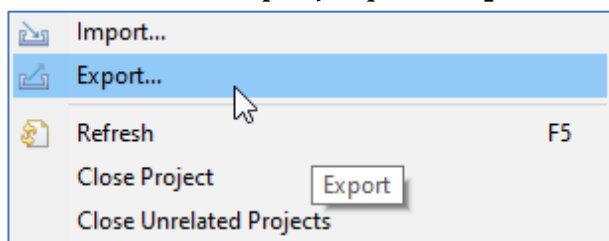
Evidemment, il peut tout à fait être livré avant !!

La procédure de rendu du projet est la suivante :

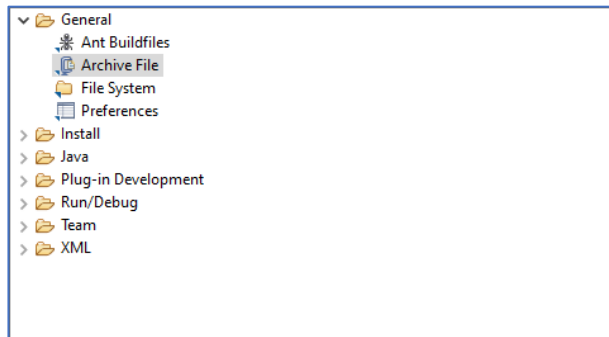
Liste des différents groupes pour export :

Groupe	N° de projet
Aaron / Léo	AgImmo1
Hind /Ashley	AgImmo2
Mohammed / Maxime / Valerian	AgImmo3
Badis / Hugo	AgImmo4
Lucas/ Bastien	AgImmo5
Yousra / Joé	AgImmo6

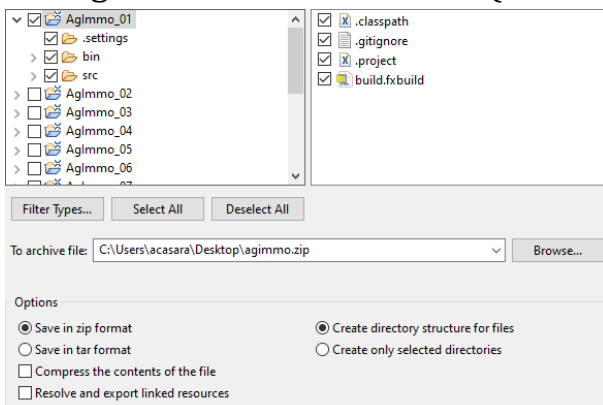
1. Clic-droit sur le projet puis **Export...**



2. Sélectionner General > Archive File



3. Configurez comme ci-dessous (nommez le zip avec votre n° d'AgImmo)



4. Le fichier doit porter le nom suivant :
Livraison_lot1_AgImmoX.zip (X=n° de votre groupe)
5. Le fichier Zip sera récupéré sur clé USB